Report for the Course on Cyber-Physical Systems and IoT Security

First mid-term project

Professional Security Corporation

Members

Emanuele Artusi | 2198545

Filippo Bellon | 2199368

Reference paper

Tractor Beam: Safe-hijacking of Consumer Drones with Adaptive GPS Spoofing

GitHub repository

`https://github.com/ProSecCorp/Part-1-CPS`

A.Y. 2025/26

Contents

1 Introduction

Unmanned Aerial Vehicles (UAVs) have progressively evolved from remotely controlled aircraft into complex autonomous Cyber-Physical Systems (CPSs). Modern UAVs are capable of executing missions with limited human intervention, autonomously following predefined trajectories, reaching waypoints and performing operations such as take-off, hovering and landing.

This increasing level of autonomy also introduces new security challenges. The correct operation of an autonomous UAV depends on the interaction between physical sensors, navigation algorithms, embedded control systems and communication interfaces. Consequently, manipulating a sensor input may affect the physical behavior of the vehicle even when the underlying flight-control software and mission remain unchanged.

Among the sensors used for autonomous navigation, Global Navigation Satellite System (GNSS) are particularly important. Position information obtained from GNSS receivers is commonly combined with inertial measurements to estimate the state of the vehicle and control its trajectory. Global Positioning System (GPS) spoofing represents a relevant threat in this context. Instead of disrupting the availability of the navigation system, a spoofing attack attempts to provide misleading position information to the receiver. If the manipulated measurements are considered valid by the flight controller, the resulting position estimate may differ from the physical position of the vehicle.

2 Background

2.1 Autonomous UAV navigation

An autonomous UAV must continuously estimate its position, velocity, attitude and other state variables in order to follow a predefined trajectory. During autonomous operation, the flight controller compares the estimated state of the vehicle with the desired state generated by the mission or path-following algorithm. The controller then generates appropriate control commands to reduce the difference between the current and desired states.

For waypoint-based navigation, the desired trajectory is defined by a sequence of geographical coordinates. The UAV attempts to reach each waypoint in sequence before continuing with the following mission item. ArduCopter provides an autonomous `AUTO` mode in which the vehicle can execute missions containing operations such as take-off, waypoint navigation, and landing.

The original Tractor Beam study also considers ArduCopter's autonomous waypoint navigation and describes how the vehicle uses its estimated position to follow the mission trajectory [1].

2.2 Flight controller

The flight controller is the computational component responsible for estimating the UAV state and generating the control commands required to maintain the desired flight behavior.

In this work, the flight controller is provided by ArduPilot ArduCopter running in Software-In-The-Loop (SITL). The use of SITL allows the complete flight-control stack to be executed without requiring physical UAV hardware. The simulated environment also provides access to internal telemetry and simulation variables, making it possible to compare the physical simulated state with the measurements available to the flight controller.

2.3 GPS and GNSS

GNSS provide positioning and timing information to receivers located on or near the Earth's surface.

GPS is one of the GNSS systems and is widely used by autonomous UAVs as a source of absolute position information. A GPS receiver estimates the position of the vehicle from measurements obtained from multiple satellites. The resulting latitude, longitude, altitude, velocity, and accuracy information is then provided to the navigation system. The use of GPS as an external source of absolute position is particularly important for autonomous flight because inertial sensors alone accumulate errors over time.

The Tractor Beam paper describes GPS as a satellite-based navigation system in which receivers estimate their position, velocity, and time from satellite information and pseudorange measurements [1].

2.4 Inertial Measurement Unit

An Inertial Measurement Unit (IMU) provides measurements from sensors such as accelerometers and gyroscopes. Unlike GPS, inertial measurements do not directly provide an absolute geographical position. Instead, the flight controller can integrate accelerations and angular rates to estimate changes in the vehicle state. However, inertial measurements contain noise and systematic errors. Consequently, the position estimate obtained exclusively from the IMU drifts over time. For this reason, autonomous UAVs commonly combine GPS and IMU measurements using sensor-fusion algorithms.

2.5 Extended Kalman Filter

ArduCopter uses an Extended Kalman Filter (EKF) to estimate the state of the vehicle by combining measurements from different sensors. The EKF continuously predicts the vehicle state using inertial measurements and compares the prediction with external measurements such as GPS. The difference between the predicted state and the GPS measurement is known as the innovation. If the innovation becomes inconsistent with the expected uncertainty of the system, the measurement may be rejected.

The Tractor Beam study specifically analyzes the ArduCopter EKF and describes how position and velocity innovations are used to detect inconsistencies between GPS and inertial measurements [1].

2.6 MAVLink

MAVLink is the communication protocol used to exchange telemetry, commands, and parameters between the flight controller and external components. In the experimental setup, MAVLink is used to monitor the simulated UAV, read GPS and navigation information, modify simulation parameters, and collect data during the experiments. The attack module is therefore external to the flight controller and interacts with the simulated vehicle through the available MAVLink interfaces.

2.7 Waypoint navigation

A waypoint mission defines a sequence of geographical positions that the UAV must reach. A simplified mission can be represented as

$$W = \{W_0, W_1, \ldots, W_n\} \tag{1}$$

where each waypoint W_i contains at least a latitude, longitude, and altitude. The flight controller continuously estimates the current vehicle position and generates control actions to move the UAV towards the active waypoint. The mission itself therefore remains unchanged during the experiments presented in this work. Any trajectory deviation is produced by modifying the position information available to the navigation system.

2.8 GPS spoofing

GPS spoofing is an attack in which an adversary generates or manipulates navigation signals so that a GPS receiver estimates an incorrect position, velocity, or time. Unlike GPS jamming, whose primary objective is to deny access to the navigation signal, spoofing attempts to preserve apparently valid navigation information while modifying its content.

The literature commonly distinguishes between hard and soft GPS spoofing.

2.8.1 Hard GPS spoofing

In hard GPS spoofing, the manipulated signal is sufficiently different from the authentic signal that the receiver may lose its original lock before acquiring the spoofed signal.

The Tractor Beam paper describes this approach as initially behaving similarly to GPS interference: the receiver may lose its lock and later acquire the spoofed signal. Such a procedure can take a significant amount of time and may activate the target's GPS failsafe mechanism [1].

Hard spoofing is therefore generally unsuitable for experiments in which the objective is to maintain continuous autonomous navigation.

2.8.2 Soft GPS spoofing

Soft GPS spoofing, instead, attempts to maintain consistency with the authentic measurements while progressively changing the apparent position. The attacker initially generates measurements close to the authentic position and gradually moves the spoofed position away from the real position. This concept is particularly relevant to the present work because the objective is to investigate whether progressive position manipulation can influence the trajectory without immediately triggering the navigation system's glitch protection.

Noh et al. describe this approach as a central component of their Strategy C, where the spoofed position is gradually deviated from the real position while attempting to preserve GPS lock [1].

2.9 GPS glitch detection and failsafe

GPS measurements cannot always be assumed to be correct. Multipath propagation, signal degradation, receiver errors, and other phenomena may produce significant position errors. ArduCopter therefore implements mechanisms for detecting anomalous GPS measurements.

A new GPS position can be compared with a position predicted from the previous position and velocity. The measurement is considered acceptable only when the displacement is compatible with predefined limits. One of the parameters relevant to this mechanism is the maximum allowed GPS glitch radius. Another constraint is related to the maximum physically reasonable position change between consecutive measurements.

The EKF provides an additional layer of protection. GPS measurements are compared with the state predicted from inertial sensors. When the difference becomes sufficiently large, the GPS measurement may be rejected and an EKF inconsistency can be reported. This behavior is fundamental for the experiments in this work because increasing the GPS offset too rapidly does not necessarily result in a corresponding change of the UAV trajectory. Instead, the flight controller may classify the measurement as anomalous and reject it. Therefore, the effective manipulation is not determined only by the absolute GPS offset, but also by its temporal evolution.

2.10 Safe hijacking

The concept of safe-hijacking was introduced by Noh et al. in the context of autonomous consumer drones. Instead of simply disrupting a UAV, safe-hijacking aims to influence its autonomous behaviour so that the vehicle can be moved away from a sensitive area [1]. The authors analyzed several consumer UAVs and classified them according to their GPS failsafe behaviour. They then designed different spoofing strategies depending on the characteristics of each class. The key observation is that a GPS manipulation cannot be designed independently from the target's navigation and failsafe mechanisms. A spoofed position that is too different from the expected position may trigger a failsafe, preventing the manipulation from producing the desired trajectory. The paper therefore distinguishes multiple strategies. In particular, Strategy C targets systems for which entering a GPS-independent failsafe mode prevents subsequent GPS manipulation from influencing the vehicle. For these systems, the spoofed GPS position must remain sufficiently consistent with the authentic measurements to avoid triggering the failsafe mechanism [1].

2.10.1 Relation to ArduCopter

An important aspect of the Tractor Beam study is its analysis of the 3DR Solo, whose software was based on ArduCopter. The authors analyzed both the EKF failure detection mechanism and the ArduCopter path-following algorithm. Their analysis showed that ArduCopter uses an intermediate target position during waypoint navigation. The Intermediate Target Position (ITP) is advanced along the trajectory toward the next waypoint, and the vehicle attempts to move towards this target position. The paper further describes a "leash" mechanism that limits how far the vehicle can deviate from the planned track before the intermediate target is updated [1].

This behaviour is particularly relevant to the present study. If the flight controller believes that the UAV is located at a position different from its physical position, the path-following algorithm may generate control actions intended to return the vehicle to the planned trajectory. Consequently, GPS manipulation can potentially influence the physical trajectory indirectly, without directly issuing movement commands to the flight controller.

2.10.2 Adaptive GPS manipulation

The strategy investigated in this work follows the same general principle of adaptive manipulation. Instead of applying a single large GPS offset, the spoofed position is progressively modified. At each iteration, the

current GPS state is observed and the next manipulation is calculated according to:

$$P_{\text{spoof}}(t) = P_{\text{real}}(t) + \Delta P(t) \tag{2}$$

where

$$\Delta P(t) = \begin{bmatrix} \Delta\phi(t) \\ \Delta\lambda(t) \end{bmatrix} \tag{3}$$

represents the accumulated GPS manipulation. The objective is to keep the change between consecutive measurements small enough to avoid immediately activating the GPS glitch protection, while progressively changing the position perceived by the navigation system. The general control loop can therefore be represented as:

$$\text{Real position} \rightarrow \text{GPS manipulation} \rightarrow \text{Estimated position} \rightarrow \text{Flight controller} \rightarrow \text{Physical trajectory} \tag{4}$$

The resulting physical trajectory changes the subsequent real position, which is then used to calculate the next spoofed position. This creates a feedback loop in which the manipulation is continuously adapted to the current state of the UAV.

3 Objectives

This work investigates the effects of controlled GPS manipulation on an autonomous UAV using ArduPilot SITL. The objective is not to modify the flight controller or the mission itself, but to study whether a UAV following a predefined waypoint mission can be progressively displaced by manipulating only its simulated GPS information.

The experimental study focuses on two increasingly complex scenarios: a constant GPS offset and an adaptive/progressive position-manipulation strategy. The latter is inspired by the concept of safe-hijacking introduced by Noh et al. [1], but is implemented in a simplified SITL environment.

The main objectives of the work are therefore:

- to understand the interaction between GPS measurements and UAV navigation;
- to analyze the GPS glitch detection mechanisms implemented by ArduCopter;
- to determine the relationship between the magnitude and evolution of a GPS offset and the resulting UAV behavior;
- to investigate whether a progressive manipulation can influence the trajectory while remaining within the limits imposed by the navigation system;
- to quantitatively evaluate the resulting deviation from the original mission.

4 ArduPilot SITL setup

First thing we want to do is setting up Ardupilot SITL. We do that by following the official guide to make it work under the Windows Subsystem for Linux (WSL) system on a Windows 11 Personal Computer (PC). This will also create a special Python environment to better manage packages and modules needed to run Ardupilot.

After setting all up we start it up by using a special command:

```
python3 /home/darckat038/uni/cps/ardupilot/Tools/autotest/sim_vehicle.py -v
↪ ArduCopter --console --map --mavproxy-args="--load-module=batteryreset
↪ --out=127.0.0.1:14550 --out=127.0.0.1:14549 --out=127.0.0.1:14548
↪ --out=127.0.0.1:14547 --out=127.0.0.1:14546 --out=127.0.0.1:14545
↪ --cmd=\"param set RTL_ALT_FINAL_M 15\""
```

With this command we are able to:

- use the ArduCopter as represented vehicle;
- show the console and the map to better visualize the drone movement and flight status;

- set alternative output ports to communicate with MAVProxy using multiple programs;
- set a final altitude for the Return to Land (RTL) mode.

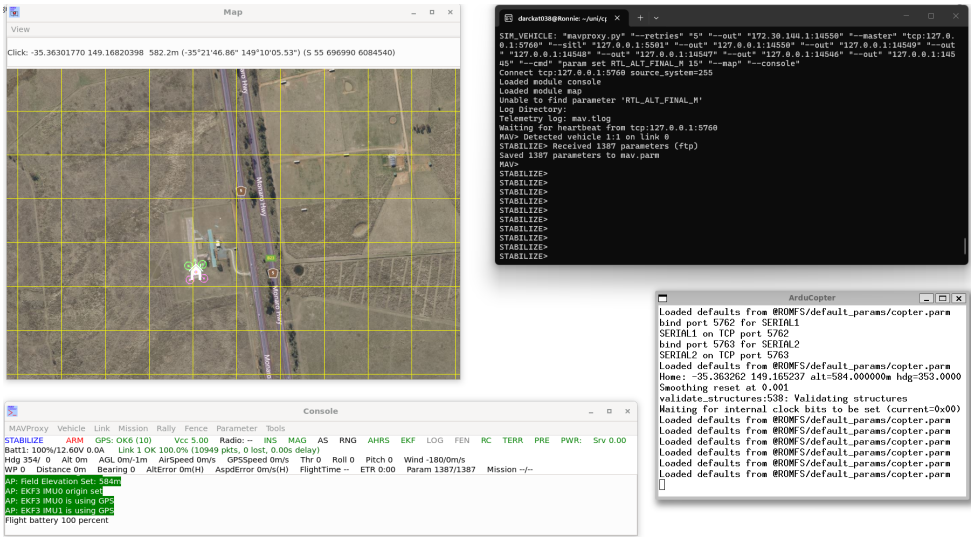


Figure 1: Final Ardupilot SITL workplace.

5 Python helper

To better achieve what we need we decided to create multiple Python classes, thus dividing responsibilities among multiple parts of the program. These files can be found in the `Source/Scripts/attacker` folder present in the project directory, each one under a specific subfolder.

5.1 GPSReader

The first and most important is the `GPSReader` class. This particularly useful Python class enabled us to continuously request the GPS position of the drone from the MAVProxy software by using a specifically exposed Application Programming Interface (API), accessible through the ports we requested in the launch command. Latitude, Longitude and Altitude were the main requested parameters.

5.2 FlightMonitor

Then comes `FlightMonitor`, which helped mainly to acquire information about the ongoing mission like the number of waypoints, the index of the landing waypoint and so on. Were also included function to gather information about events and status changes which helped us better understand the thresholds and limits supported by the flight controller.

5.3 GlitchController

The `GlitchController` class, as the name says, had the obvious goal of setting and getting the desired glitch parameters through an API endpoint.

5.4 FlightLogger

To `FlightLogger` class has then been given the task of logging every step of the GPS of the drone, along with the currently set glitch and any events that may occur during the spoofing period.

5.5 Other utils

5.5.1 Distance

A `distance` method has been written to compute the distance between two coordinates and help with the progressive/adaptive glitch computations and development.

5.5.2 Plotting

A `plots` file has been written to plot the logged flight data and visualize the drone trajectory and the glitch injection to better understand the effects of the constant glitch injection on the drone flight. A `plot_diff` file has also been written to plot the difference between the logged flight data and the original mission trajectory to better visualize the effects and the deviation from the original path, in particular for the adaptive/progressive glitch injection.

6 Experiments

As we said before, we wanted to test two different types of GPS spoofing:

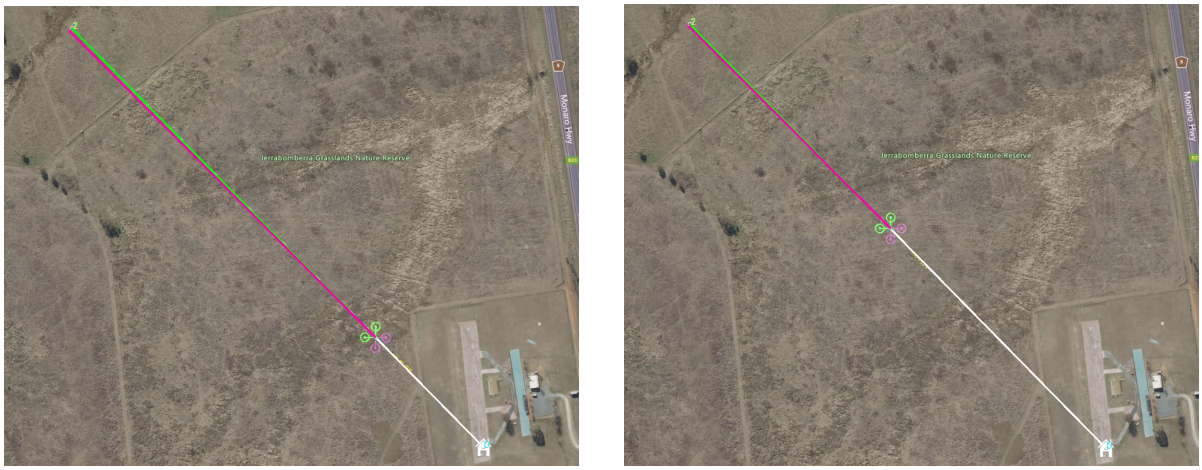
1. constant GPS spoofing, where we will set a fixed offset and observe how the software behaves. We will start with a small value first, and then we will test a bigger one;
2. progressive/adaptive GPS spoofing, in which we will increment the offset as the drone moves along a specific multi-point trajectory, adapting the increment in relation to the speed and desired spoofing direction.

6.1 Fixed/Constant GPS spoofing

Let's first test GPS spoofing with a fixed offset approach, where we discuss both the small one and the big one. The main file used for this test is located at `Source/Scripts/attacker/spoofer/` and is named `constant_spoofer.py`.

6.1.1 Small offset

The offset that has been tested is a **0.00003°** offset, which is applied to the **x-axis**. This value was chosen because, after several tests, it was found to be the maximum value at which the system does not detect a change large enough to cause the flight controller to go into error mode and for the GPS to glitch.



(a) The small glitch is applied.

(b) The small glitch is reset.

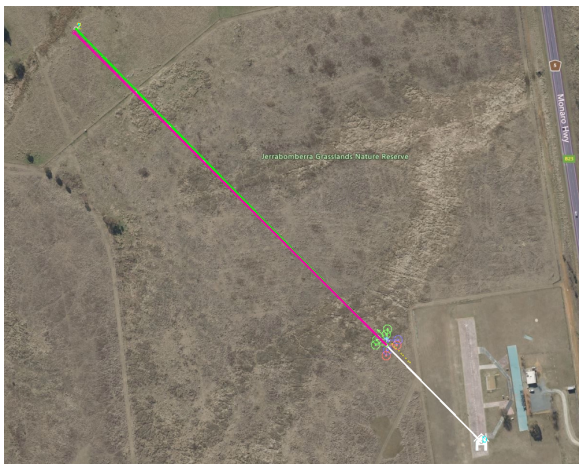
Figure 2: Map view of constant small glitch test.

As shown in the figure, the glitch is introduced after the drone has entered `AUTO` mode and started following the predefined mission trajectory. In particular, the injection begins once the drone reaches an altitude of approximately 10 meters, as shown in Figure 2a. The glitch is maintained for approximately 15 seconds. During this interval, a small and almost imperceptible deviation from the original trajectory can be observed. This change is highlighted by the yellow dashed line, which represents the altered trajectory resulting from the injected GPS offset.

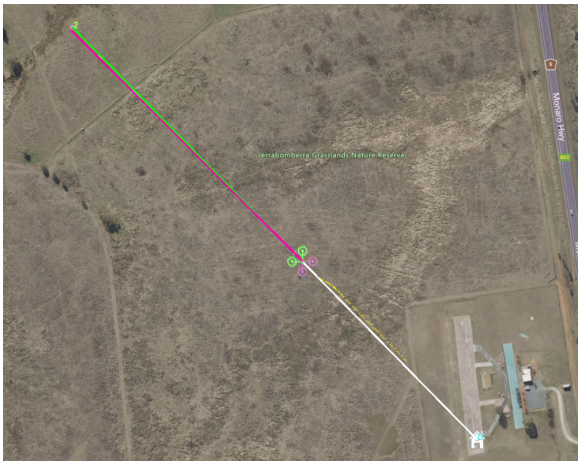
After the attack time has passed, the glitch is reset to zero. Following the reset, the trajectory exhibits another small variation, also visible through the yellow dashed line in Figure 2b. In this case, however, the resulting position discrepancy remains sufficiently small that it does not trigger a GPS glitch warning from the flight controller.

6.1.2 Big offset

The offset has been increased to **0.00006°** and applied again to the **x-axis**, this time with the purpose of observing the behavior of the software and, in particular, the drone GPS.



(a) The big glitch is applied.

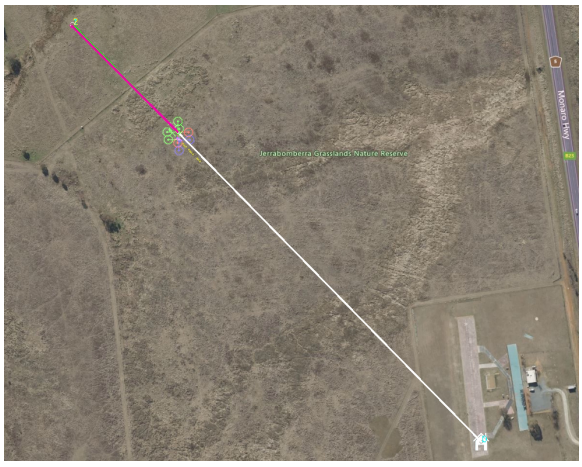


(b) The big glitch is still applied but the GPS clears it.

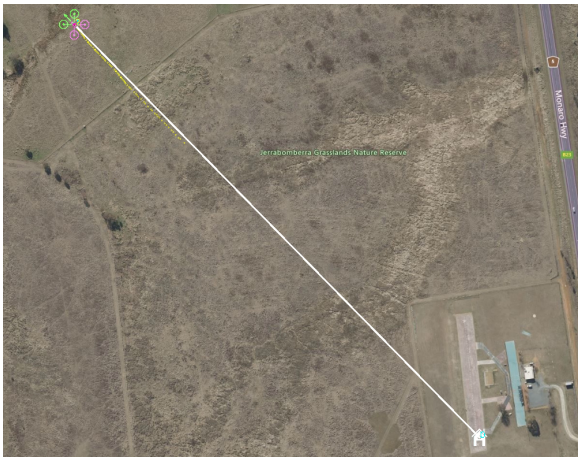
Figure 3: Map view of constant big glitch applied.

As soon as the glitch is applied, the drone icon appears to be duplicated in the ground station, with a second trajectory developing in parallel to the original one (Figure 3a). This behavior reflects the discrepancy between the position reported by the GPS and the position estimated by the flight controller from the other available sensors. At the same time, ArduPilot reports a `GPS Glitch or Compass error`, indicating that the inconsistency has been detected by the navigation system.

As the glitch persists, the flight controller continues to reject or compensate for the inconsistent GPS measurements. After a certain period, the GPS-derived position is realigned with the estimated vehicle position, and the two displayed trajectories progressively converge again. During this transition, ArduPilot reports an `EKF3 lane switch` followed by an `EKF3 primary changed` event, indicating a change in the EKF3 estimator configuration. Once the inconsistency has been resolved, the system reports `GPS Glitch cleared`, marking the end of the detected GPS glitch (Figure 3b).



(a) The big glitch is removed.



(b) The big glitch is removed and the GPS clears the mismatch.

Figure 4: Map view of constant big glitch removed.

A similar transition can be observed when the injected glitch is removed. When the glitch value is reset to zero, the GPS position changes back towards the original position, again creating a discrepancy with the position estimated by the flight controller (Figure 4a). As a result, the two trajectories temporarily diverge once more before progressively converging. The flight controller, consequently, performs the necessary estimator

adjustments to accommodate the change in the GPS measurement, after which the GPS position is once again considered consistent, and the `GPS Glitch cleared` message is generated (Figure 4b).

6.2 Progressive/Adaptive GPS spoofing

As the final test we then proceded with a progressive GPS spoofing approach, where the offset is incrementally increased as the drone moves along a predefined multi-point trajectory. The goal of this test was to observe how the flight controller adapts to gradually changing GPS measurements and whether it can maintain stable flight under these conditions without going into an error state.

The main file used for this test is located inside `Source/Scripts/attacker/spoofer/` folder and is called `adaptive-progressive_spoofer.py`.

First of all the program asks the user to input the desired spoofing direction, which can be either **North**, **South**, **East**, **West** or their combinations. Then sets an offset increment to the glitch of 0.00001° and applies it to the x-axis and/or y-axis depending on the chosen direction (for example in this experiment we choose to spoof North-East so the program will apply a -0.00001° offset increment to the x-axis and a -0.00001° offset increment to the y-axis).

In parallel to the offset increment, every 0.5 seconds a second thread will log the GPS position, the x-glitch, the y-glitch and any events reported by the flight controller. This logging process will continue until the drone reaches the last waypoint and will stop some time after the landing procedure is started.

Given that the test mission is composed of three waypoints and a final landing on the last one, the program has been designed to increment the glitch only when the drone has been away from the previous waypoint for a couple of seconds and is more than 50 meters away from the next waypoint, in order to make it reach an ideal speed to avoid detection and errors from the flight controller and the GPS.

(a) Arrival position.

(b) Glitching position.

(c) Final realigned position.

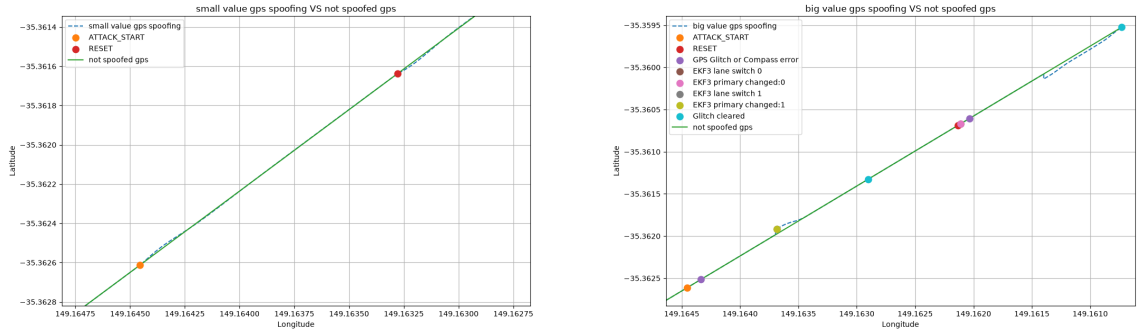
Figure 5: Final map view of the adaptaptive/progressive GPS spoofing.

As the drone reaches the last waypoint, it will start the landing procedure while the glitch is still being applied but not incremented (Figure 5a). After the landing procedure is completed, the glitch is then reset to zero and the incoherence between the believed GPS position and the real one can be seen as the drone icon is duplicated in the ground station (Figure 5b). After a few seconds, the GPS position is realigned with the

estimated vehicle position and the two displayed positions are aligned again showing the effectiveness of the spoofing (Figure 5c).

7 Results and discussion

7.1 Fixed GPS spoofing



(a) Small glitch test.

(b) Big glitch test.

Figure 6: Graphs of the fluctuations in the GPS position during the fixed GPS spoofing tests.

As can be seen in both figures, the introduction of the glitch is clearly visible in the graph, with a sudden jump in the GPS position corresponding to the applied offset. In the case of the small glitch (Figure 6a), the jump is relatively minor, and the GPS position quickly returns to its original trajectory. In contrast, the big glitch (Figure 6b) shows a more pronounced jump both when it is introduced and when it is reset, followed by messages stating the detection of a GPS glitch and the subsequent realignment of the GPS position with the estimated vehicle position. Notable is the presence of EKF3 lane switch and EKF3 primary changed events, which indicate a change in the EKF3 estimator configuration as the system compensates for the inconsistent GPS measurements.

7.2 Progressive/Adaptive GPS spoofing

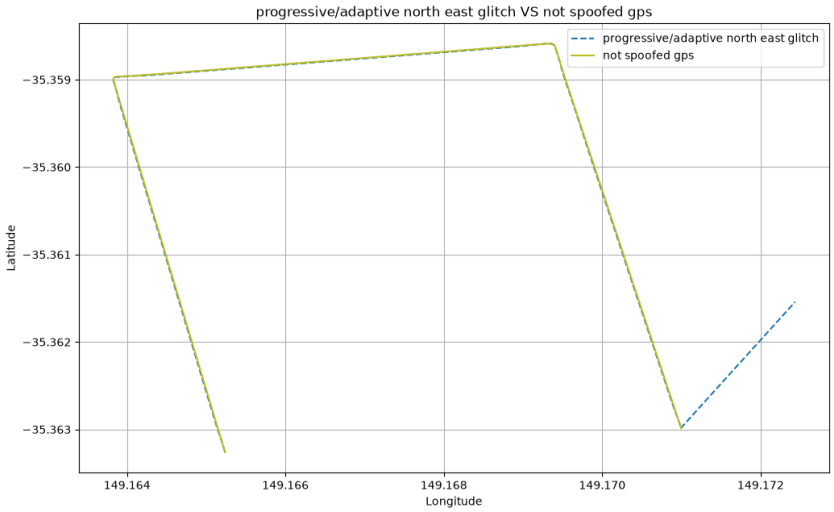


Figure 7: Graph of the fluctuations in the GPS position during the progressive/adaptive GPS spoofing test.

The graph in Figure 7 compares the GPS trajectory obtained during the progressive spoofing test with the trajectory recorded in the absence of spoofing. Throughout most of the experiment, the two trajectories overlap almost completely, indicating that the progressive increase of the applied glitch does not produce a detectable GPS glitch or compass error. The flight controller therefore continues to accept the GPS measurements and the vehicle follows the intended trajectory without triggering the previously observed glitch protection mechanisms.

A significant difference can instead be observed at the end of the experiment, when the applied glitch is reset to zero. This sudden removal of the offset causes an abrupt discontinuity in the spoofed GPS position, visible as the dashed blue segment extending towards the right-hand side of the figure. This segment should be interpreted in the opposite direction, from right to left: the GPS position first jumps away from the previously followed trajectory and then progressively returns towards the final position of the non-spoofed trajectory as the GPS position is realigned with the estimated vehicle position.

This behavior is consistent with the sudden removal of the accumulated GPS offset. While the progressive spoofing phase allows the GPS position to remain sufficiently close to the position expected by the flight controller, resetting the glitch instantaneously removes the accumulated displacement. The resulting discrepancy is therefore much larger and produces the visible transient realignment at the end of the experiment.

Acronyms

API Application Programming Interface. 6

CPS Cyber-Physical System. 2

EKF Extended Kalman Filter. 3, 4, 8, 10

GNSS Global Navigation Satellite System. 2

GPS Global Positioning System. 2–11

IMU Inertial Measurement Unit. 3

ITP Intermediate Target Position. 4

PC Personal Computer. 5

RTL Return to Land. 6

SITL Software-In-The-Loop. 2, 5, 6

UAV Unmanned Aerial Vehicle. 2–5

WSL Windows Subsystem for Linux. 5

References

[1] Juhwan Noh, Yujin Kwon, Yunmok Son, Hocheol Shin, Dohyun Kim, Jaeyeong Choi, and Yongdae Kim. Tractor beam: Safe-hijacking of consumer drones with adaptive gps spoofing. *ACM Trans. Priv. Secur.*, 22(2), April 2019.